The Erronatus Blueprint

AI Automation with OpenClaw

BUSINESS EDITION

© 2026 Erronatus · erronatus.com
All rights reserved. Do not redistribute.

Table of Contents

1. Chapter 1: The Foundation
2. Chapter 2: Infrastructure
3. Chapter 3: Your Command Interface
4. Chapter 4: Engine Configuration
5. Chapter 5: API Integration
6. Chapter 6: Memory Systems
7. Chapter 7: Automation
8. Chapter 8: Deployment
9. Appendix A: Troubleshooting & FAQ
10. Chapter 9: Multi-Engine Routing
11. Chapter 10: The Full API Toolchain
12. Chapter 11: Trading Automation
13. Chapter 12: Advanced Memory
14. Chapter 13: Custom Skill Development
15. Chapter 14: Email Automation
16. Chapter 15: Payment Integration
17. Chapter 16: 24/7 Operations

Chapter 1: The Foundation

Why AI Automation Changes Everything

The Shift That's Already Happened

In 2024, AI could answer questions. In 2025, AI could write code. In 2026, AI can operate autonomously — monitoring data, executing tasks, managing workflows, and making decisions based on rules you define.

The difference between a chatbot and an AI automation system is the difference between a calculator and a spreadsheet. One answers when asked. The other runs continuously, processing data, applying logic, and producing results without human intervention.

This guide teaches you to build the second kind.

What AI Automation Actually Means

When we say "AI automation," we don't mean:

- ❌ A chatbot that answers customer service tickets
- ❌ A writing tool that generates blog posts
- ❌ A code assistant that autocompletes functions

We mean:

- ✅ An AI agent that monitors your email inbox every 2 hours and summarizes anything urgent
- ✅ An AI agent that checks stock market indicators on a schedule and alerts you when signals cross thresholds
- ✅ An AI agent that maintains persistent memory across sessions, remembering your projects, preferences, and priorities
- ✅ An AI agent that selects the cheapest capable AI model for each task automatically, optimizing your costs
- ✅ An AI agent that runs 24/7 on your own infrastructure, under your control

This is not theoretical. By the end of this guide, every item on that list will be running on your machine.

The Architecture

Every AI automation system has five layers:

```
┌──────────────────────────────┐
│       5. AUTOMATION          │ ← Cron jobs, scheduled tasks
├──────────────────────────────┤
│       4. MEMORY              │ ← Persistent context across sessions
├──────────────────────────────┤
│       3. TOOLS & APIs        │ ← External service connections
├──────────────────────────────┤
│       2. AI ENGINE           │ ← Language models (the "brain")
├──────────────────────────────┤
│       1. INFRASTRUCTURE      │ ← OpenClaw gateway + channels
└──────────────────────────────┘
```

Layer 1: Infrastructure is the foundation. OpenClaw runs as a gateway on your machine, routing messages between you and AI models. It handles authentication, context management, tool execution, and session persistence.

Layer 2: AI Engine is the brain. You configure which AI models to use — from free models for simple tasks to premium models for complex reasoning. OpenClaw supports dozens of providers through a single interface.

Layer 3: Tools & APIs are the hands. Your AI can read files, execute commands, search the web, call HTTP APIs, control a browser, and interact with external services. Each API you connect multiplies what your AI can do.

Layer 4: Memory is the continuity. Without memory, your AI wakes up with amnesia every session. With memory, it remembers your name, your projects, your preferences, what happened yesterday, and what needs to happen tomorrow.

Layer 5: Automation is the autonomy. Cron jobs execute tasks on a schedule without you asking. Your AI checks data, sends reports, monitors systems, and executes workflows — automatically.

Each chapter of this guide builds one layer. By the end, you'll have all five working together.

The Cost Reality

Let's address the elephant in the room: how much does this cost to run?

Infrastructure costs: $0 OpenClaw is open source. It runs on your existing machine. No cloud servers required for personal use.

AI model costs: $3-15/day (with smart routing) This is where the money goes. Every time your AI processes a message, it costs a fraction of a cent to a few cents depending on the model. The key insight is *engine routing* — using free models for simple tasks and premium models only when needed.

Here's a realistic daily breakdown:

Task Type	Model	Cost per call	Daily calls	Daily cost
Status checks	Gemini Flash Lite	$0.00	20	$0.00
Summaries, parsing	DeepSeek V3	$0.001	30	$0.03
Research, coding	Claude Sonnet	$0.01-0.05	15	$0.30
Complex analysis	Claude Opus	$0.10-0.30	3	$0.60
Total			**68**	**~$0.93**

Most personal users spend $1-5/day. Heavy users with trading bots and monitoring might hit $10-15/day. Compare that to hiring a virtual assistant ($15-30/hour) or subscribing to enterprise automation tools ($200-500/month).

API costs: Varies Most APIs we'll use have generous free tiers. Alpha Vantage gives you 25 free calls/day. NewsAPI gives you 100. Alpaca paper trading is free. Supabase has a free tier. The only API with meaningful costs is email (Resend charges $0.001/email after your free 100/month).

Bottom line: A fully operational AI automation system costs less per day than your morning coffee.

The Compound Effect

Here's what most people miss about AI automation: it compounds.

On Day 1, your AI can answer questions through Telegram. Useful, but not life-changing.

On Day 7, it remembers your previous conversations and has access to 5 APIs. It can check news, pull market data, and search the web — without you leaving your messaging app.

On Day 30, it's been running cron jobs for weeks. It has a month of memory. It knows your projects, your communication style, your priorities. It sends you morning briefings automatically. It alerts you when important things happen.

On Day 90, you can't imagine working without it. It's become an extension of your workflow — handling the repetitive, the routine, and the time-sensitive while you focus on the creative, the strategic, and the important.

The system gets more valuable over time because:

1. **Memory accumulates** — more context = better decisions
2. **Automations stack** — each new cron job adds value without adding effort
3. **APIs multiply** — each new connection creates new possibilities
4. **Your AI learns your patterns** — it gets better at anticipating what you need

This is not a tool you use. It's a system you build. And it pays dividends forever.

Who This Guide Is For

This guide assumes:

- You can install software on your computer
- You can edit text files
- You can follow step-by-step instructions
- You have basic familiarity with the command line (we'll teach what you need)

This guide does NOT require:

- Programming experience (though it helps)
- AI/ML knowledge
- DevOps experience
- A computer science degree

If you can install an app and edit a document, you can build an AI automation system. Let's start.

Next Chapter: Infrastructure — Installing & Configuring OpenClaw →

Chapter 2: Infrastructure

Installing & Configuring OpenClaw

What We're Building

By the end of this chapter, you'll have:

- OpenClaw installed on your machine
- A workspace directory initialized with all configuration files
- The gateway running and ready to accept connections
- Your first AI model connected and responding

Time required: 30-45 minutes.

Step 1: Install Node.js

OpenClaw runs on Node.js. If you don't have it installed:

Windows:

1. Visit nodejs.org
2. Download the LTS version (20.x or higher)
3. Run the installer, accept all defaults
4. Open PowerShell and verify: `node --version`

macOS:

```
# Using Homebrew (recommended)
brew install node

# Verify
node --version
```

Linux (Ubuntu/Debian):

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs
node --version
```

You need Node.js 20 or higher. If `node --version` shows 20.x or above, you're good.

Step 2: Install OpenClaw

Open your terminal (PowerShell on Windows, Terminal on macOS/Linux):

```
npm install -g openclaw
```

This installs OpenClaw globally. Verify the installation:

```
openclaw --version
```

You should see the version number (2026.3.x or similar).

Step 3: Initialize Your Workspace

Your workspace is where OpenClaw stores all configuration, memory, and project files. Initialize it:

```
openclaw init
```

The initialization wizard will ask you several questions:

1. **Workspace location** — Accept the default (`~/.openclaw/workspace/`) or choose your own
2. **Channel** — Select Telegram or Discord (we'll configure this in Chapter 3)
3. **AI Provider** — Select your provider (OpenRouter recommended for beginners)
4. **API Key** — Enter your AI provider API key

> ***Getting an API Key:*** *If you don't have an AI provider API key yet:*
>
> *1. Go to openrouter.ai and create a free account*
>
> *2. Navigate to Keys → Create Key*

> *3. Copy the key (starts with `sk-or-v1-`)*
>
> *4. Add $5-10 of credit to start*
>
> *OpenRouter gives you access to dozens of models (GPT-4, Claude, Gemini, DeepSeek, Llama, and more) through a single API key. This is the easiest way to get started.*

After initialization, your workspace will contain:

```
~/.openclaw/
├── config.yaml          ← Gateway configuration
├── workspace/
│   ├── AGENTS.md        ← AI behavior rules
│   ├── SOUL.md          ← AI personality & guidelines
│   ├── USER.md          ← Information about you
│   ├── MEMORY.md        ← Long-term memory
│   ├── TOOLS.md         ← Local environment notes
│   └── memory/          ← Daily memory files
└── .env                 ← API keys and secrets
```

Step 4: Configure Your AI's Identity

This is where OpenClaw differs from every other AI tool. Your AI reads these files every session to understand who it is and who you are.

Edit `SOUL.md` — This defines your AI's personality:

```
# SOUL.md - Who I Am

I'm your personal AI automation assistant. I help you build
systems, automate tasks, and manage information efficiently.

## Core Principles
- Be direct and concise
- Solve problems, don't just describe them
- Ask before taking external actions
- Remember context across sessions

## Communication Style
- Clear, professional, no filler
- Use bullet points for lists
- Provide code examples when relevant
```

Edit `USER.md` — This tells your AI about you:

```
# USER.md - About My Human
```

```
- **Name:** [Your name]
- **Timezone:** [Your timezone, e.g., America/New_York]
- **Goals:** [What you want to automate]
- **Preferences:** [Communication style, topics of interest]
```

Edit `AGENTS.md` — This defines operational rules. The default is well-crafted, but you can customize:

```
# AGENTS.md

## Every Session
1. Read SOUL.md — who you are
2. Read USER.md — who you're helping
3. Read recent memory files for context

## Memory
- Write daily notes to memory/YYYY-MM-DD.md
- Update MEMORY.md with important long-term information
- Capture decisions, context, lessons learned

## Safety
- Don't run destructive commands without asking
- Ask before sending emails or public posts
- Keep private information private
```

> ***Pro Tip:*** *Don't overthink these files initially. Start with the basics and refine them as you learn what works. Your AI will read them every session, so changes take effect immediately.*

Step 5: Start the Gateway

The gateway is OpenClaw's core process. It handles all communication, model routing, and tool execution.

```
openclaw gateway start
```

You should see output like:

```
✓ Gateway started on port 18789
✓ Workspace loaded: ~/.openclaw/workspace/
✓ Model: anthropic/claude-sonnet-4-20250514
✓ Channel: telegram (pending configuration)
```

Check the status at any time:

```
openclaw status
```

This shows:

- Gateway version and uptime
- Active model and token usage
- Connected channels
- Session information
- Cost tracking

Step 6: Verify Everything Works

Before we connect a messaging channel (next chapter), let's verify the AI is responding:

```
openclaw gateway status
```

If everything is green, your infrastructure is ready. The gateway is running, your workspace is configured, and your AI model is connected.

Understanding the Configuration

Your `config.yaml` file controls everything. Here are the key settings:

```yaml
# Model Configuration
model: anthropic/claude-sonnet-4-20250514    # Default AI model
thinking: adaptive                            # Reasoning mode

# Workspace
workspace: ~/.openclaw/workspace/

# Gateway
gateway:
  port: 18789

# Channel (configured in Chapter 3)
channels:
  telegram:
    token: ""    # Your bot token goes here
```

You can edit this file directly or use:

```
openclaw gateway config
```

Troubleshooting

"Command not found" after installing:

- Windows: Close and reopen PowerShell
- macOS/Linux: Run `source ~/.bashrc` or open a new terminal

"Invalid API key" error:

- Double-check your key has no extra spaces
- Verify the key is active at your provider's dashboard
- Make sure you have credit/balance on the account

Gateway won't start:

- Check if port 18789 is available: `netstat -an | grep 18789`
- Try a different port in config.yaml
- Check logs: `openclaw gateway logs`

"Workspace not found" error:

- Run `openclaw init` again
- Verify the path in config.yaml matches your workspace location

What You've Built

At the end of this chapter, you have:

✅ Node.js installed and verified ✅ OpenClaw installed globally ✅ Workspace initialized with identity files ✅ AI personality and user information configured ✅ Gateway running and connected to an AI model

Your AI exists. It has an identity, it knows who you are, and it's ready to work. But right now, the only way to talk to it is through the command line.

In the next chapter, we'll give it a proper communication channel — so you can message it from your phone, your desktop, or anywhere in the world.

Next Chapter: Your Command Interface — Telegram & Discord Setup →

Chapter 3: Your Command Interface

Telegram & Discord Setup

Why a Messaging Channel Matters

Your AI gateway is running. Your models are connected. But right now, interacting with it requires sitting at your computer and using the command line.

A messaging channel changes everything:

- **Message your AI from your phone** while walking, commuting, or lying in bed
- **Receive proactive alerts** when your AI detects something important
- **Get scheduled reports** delivered to your pocket automatically
- **Voice messages** — speak your requests instead of typing them
- **Rich formatting** — code blocks, images, files, inline buttons

We support both Telegram and Discord. Pick whichever you prefer. This chapter covers both.

Option A: Telegram Setup

Telegram is our recommended channel. It's fast, supports rich formatting, works on every platform, and the bot API is excellent.

Step 1: Create Your Bot

1. Open Telegram and search for `@BotFather`
2. Send `/newbot`
3. Choose a display name (e.g., "My AI Assistant")
4. Choose a username (must end in `bot`, e.g., `myai_automation_bot`)

5. BotFather will give you an API token — **copy it immediately**

The token looks like: `7123456789:AAH1234567890abcdefghijklmnop`

> ⚠️ ***Security:*** *This token controls your bot. Never share it publicly, commit it to git, or post it anywhere.*

Step 2: Configure the Bot

Send these commands to @BotFather to customize your bot:

```
/setdescription - Set a description for your bot
/setabouttext - Set the about text
/setuserpic - Upload a profile picture
/setcommands - Set the command menu
```

For the command menu, send:

```
status - Check system status
new - Start a new session
reset - Reset conversation context
```

Step 3: Connect to OpenClaw

Add your Telegram bot token to the OpenClaw configuration:

```
openclaw gateway config
```

Navigate to the Telegram channel section and enter your bot token. Or edit `config.yaml` directly:

```yaml
channels:
  telegram:
    token: "7123456789:AAH1234567890abcdefghijklmnop"
```

Step 4: Start Talking

Restart the gateway to apply changes:

```
openclaw gateway restart
```

Open Telegram, find your bot, and send a message:

```
Hello! What can you do?
```

Your AI should respond within seconds. If it does — congratulations. You now have a personal AI assistant in your pocket.

Step 5: Secure Your Bot

By default, anyone who finds your bot can message it. Lock it down:

In your `config.yaml`, add authorized senders:

```yaml
channels:
  telegram:
    token: "your-token"
    allowedSenders:
      - 123456789    # Your Telegram user ID
```

To find your Telegram user ID:

1. Message @userinfobot on Telegram
2. It will reply with your numeric ID
3. Add that number to `allowedSenders`

Now only you can interact with your AI.

Option B: Discord Setup

Discord is ideal if you're already in Discord communities or want your AI in a server with other people.

Step 1: Create a Discord Application

1. Go to discord.com/developers/applications
2. Click "New Application" → name it → Create
3. Go to the "Bot" section in the left sidebar
4. Click "Add Bot" → Confirm
5. Under the bot's token section, click "Reset Token" → Copy it

Step 2: Configure Bot Permissions

In the "Bot" section:

- Enable "Message Content Intent" (required for reading messages)
- Enable "Server Members Intent" if you want member info

In the "OAuth2" → "URL Generator" section:

- Select scopes: `bot`, `applications.commands`
- Select permissions: `Send Messages`, `Read Message History`, `Embed Links`, `Attach Files`, `Add Reactions`
- Copy the generated URL and open it to invite the bot to your server

Step 3: Connect to OpenClaw

Edit your `config.yaml`:

```yaml
channels:
  discord:
    token: "your-discord-bot-token"
    allowedGuilds:
      - "your-server-id"    # Right-click server → Copy ID
```

Step 4: Start and Test

```
openclaw gateway restart
```

Go to your Discord server and message the bot or mention it:

```
@YourBot What's the weather today?
```

Testing Your Setup

Regardless of which channel you chose, run through these tests:

Test 1: Basic Response

Send: "What is 2 + 2?" Expected: A clear answer (probably with some personality based on your SOUL.md)

Test 2: Memory Check

Send: "My favorite color is blue. Remember that." Then send: "What's my favorite color?" Expected: It should remember "blue" within the same session

Test 3: Tool Usage

Send: "Search the web for today's top tech news" Expected: Your AI searches the web and returns results

Test 4: File Access

Send: "Read my SOUL.md file and tell me what it says" Expected: It reads the file from your workspace and summarizes it

Test 5: Status Check

Send: "/status" Expected: A status card showing model, tokens used, session info

If all five tests pass, your command interface is fully operational.

What's Possible Now

With a messaging channel connected, you can:

📱 **Message from anywhere** — Phone, tablet, desktop, web browser 🔔 **Receive alerts** — Your AI can proactively send you messages 📎 **Share files** — Send documents, images, or data for your AI to process 🎤 **Voice messages** — Speak your requests (Telegram supports this natively) 🔘 **Inline buttons** — Your AI can present clickable options 📊 **Rich formatting** — Code blocks, bold, italic, links, and more

This is your command and control channel. Everything we build from here — APIs, memory, automation — feeds through this interface.

Next Chapter: Engine Configuration — Choosing Your AI Models →

Chapter 4: Engine Configuration

Choosing Your AI Models

Why Model Selection Matters

Not all AI models are created equal. Some are fast and cheap. Others are slow and brilliant. The key to an efficient automation system is using the **right model for each task**.

Running everything through Claude Opus (the most powerful model available) is like hiring a brain surgeon to answer your phone calls. It works, but it's expensive and unnecessary.

Smart engine configuration means:

- **Free models** handle quick lookups and status checks
- **Low-cost models** handle parsing, summaries, and routine automation
- **Mid-tier models** handle research, coding, and analysis
- **Premium models** handle complex strategy and architecture only when needed

The result: you get premium intelligence when it matters and spend almost nothing on routine tasks.

Understanding AI Providers

OpenClaw supports multiple AI providers through a unified interface:

Provider	Key Models	Strength	Cost
OpenRouter	Access to 100+ models	One key, many models	Varies
Anthropic	Claude Sonnet, Opus	Best reasoning	$$ - $$$
OpenAI	GPT-4o, GPT-4o-mini	Versatile, fast	$ - $$
Google	Gemini Flash, Pro	Free tier available	Free - $$

DeepSeek	DeepSeek V3	Excellent value	$

Our recommendation for beginners: Start with OpenRouter. One API key gives you access to models from every provider. You can experiment with different models and find what works best for your use cases.

Configuring Your Default Model

Your default model handles most interactions. Set it in `config.yaml`:

```
model: anthropic/claude-sonnet-4-20250514
```

Claude Sonnet is the best all-around choice:

- Excellent at coding, research, and analysis
- Good at following complex instructions
- Reasonable cost ($3 per million input tokens)
- Fast enough for real-time conversation

Setting Up Model Aliases

OpenClaw supports aliases for quick model switching. Add these to your configuration:

```
aliases:
  flash: openrouter/google/gemini-2.0-flash-lite-001    # Free
  deepseek: openrouter/deepseek/deepseek-v3.2           # Budget
  sonnet: anthropic/claude-sonnet-4-20250514              # Standard
  opus: anthropic/claude-opus-4-6                         # Premium
  gpt: openai/gpt-4o                                     # Alternative
  mini: openai/gpt-4o-mini                                # Budget alternative
```

Now you can switch models on the fly:

- `/model flash` — Switch to the free model for quick tasks
- `/model sonnet` — Switch back to standard for complex work
- `/model opus` — Engage premium reasoning for important decisions

Model Selection Guide

Here's how to think about which model to use:

Use Flash (Free) when:

- Checking the weather
- Simple math or conversions
- Quick factual lookups
- "What time is it in Tokyo?"
- Status checks and system monitoring

Use DeepSeek ($) when:

- Summarizing articles or documents
- Parsing data into structured formats
- Running cron job automations
- Basic code generation
- Email drafting

Use Sonnet ($$) when:

- Writing code for projects
- Research and analysis
- Debugging complex issues
- Blog writing and content creation
- Planning and project management

Use Opus ($$$) when:

- Architectural decisions for large projects
- Complex financial analysis
- Strategic planning and business decisions
- Multi-step reasoning problems
- Anything where the cost of being wrong exceeds the cost of the model

Cost Management

Set a daily budget to prevent runaway spending:

You can track your spending with:

```
/status
```

This shows token usage, model distribution, and estimated cost for the current session.

Cost optimization tips:

1. Start every session on your default model (Sonnet)
2. Switch to Flash for quick lookups mid-conversation
3. Only invoke Opus when you explicitly need deep reasoning
4. Use cron jobs on DeepSeek — they run unattended and don't need premium models
5. Monitor your daily spend and adjust routing as needed

Testing Your Configuration

Let's verify your models are working:

Test 1: Default Model Send a message to your AI: "Explain quantum computing in 3 sentences." Expected: A thoughtful, well-structured response from your default model.

Test 2: Model Switching Send: "/model flash" Then: "What's 2 + 2?" Expected: A quick response from the free model.

Test 3: Switch Back Send: "/model sonnet" Then: "Write a Python function that calculates compound interest." Expected: A detailed, working code response from Sonnet.

What You've Configured

At the end of this chapter, you have:

✅ A default AI model set for general use ✅ Model aliases configured for quick switching ✅ Understanding of when to use each model tier ✅ Cost management awareness ✅ Multiple models tested and verified

Your AI now has flexible intelligence — it can be economical for routine tasks and brilliant for complex ones. In the next chapter, we'll give it the ability to interact with the outside world through APIs.

Next Chapter: API Integration — Connecting Your First 5 Services →

Chapter 5: API Integration

Connecting Your First 5 Services

Why APIs Matter

Right now, your AI can think, communicate, and remember. But it can't *do* things in the real world. It can't check stock prices, send emails, query databases, or deploy websites.

APIs change that. Every API you connect is a new capability — a new hand your AI can use to interact with the outside world.

In this chapter, we'll connect 5 essential services that cover the most common automation needs:

1. **Brave Search** — Web search capability
2. **Alpha Vantage** — Financial market data
3. **NewsAPI** — Real-time news headlines
4. **Supabase** — Cloud database
5. **GitHub** — Code repository access

How API Keys Work

Every external service requires authentication — proof that you're authorized to use it. This comes in the form of API keys: unique strings that identify your account.

API keys are stored in your `.env` file at `~/.openclaw/.env`:

```
# .env — API Keys and Secrets
# Never commit this file to git. Never share these keys.

OPENROUTER_API_KEY=sk-or-v1-your-key-here
BRAVE_SEARCH_KEY=your-brave-key-here
```

OpenClaw loads these automatically. Your AI can use them without ever seeing the raw keys.

> ⚠️ ***Security Rules:***
>
> *1. Never commit `.env` to git (it's in `.gitignore` by default)*
> *2. Never paste API keys in chat messages*
> *3. Never share keys with anyone*
> *4. Rotate keys immediately if you suspect a leak*
> *5. Use the minimum permissions each key needs*

Service 1: Brave Search

What it does: Lets your AI search the web for real-time information.

Get your key:

1. Go to brave.com/search/api
2. Create a free account
3. Subscribe to the Free plan (2,000 queries/month)
4. Copy your API key

Add to `.env`:

```
BRAVE_SEARCH_KEY=BSAzrul5b7JTv7oFjLR9NbIrLpdtMy8
```

Test it: Send to your AI: "Search the web for the latest AI news today"

Your AI will use the Brave Search API to find and summarize recent results. This is incredibly powerful — your AI now has access to real-time information that goes beyond its training data.

Service 2: Alpha Vantage

What it does: Provides stock market data, technical indicators (RSI, MACD, moving averages), and financial metrics.

Get your key:

1. Go to alphavantage.co
2. Click "Get Your Free API Key"
3. Fill in the form and submit
4. Copy your key (free tier: 25 requests/day)

Add to `.env`:

```
ALPHA_VANTAGE_KEY=your-key-here
```

Test it: Ask your AI: "What's the current RSI for AAPL?"

Your AI will call the Alpha Vantage API, retrieve the RSI indicator, and analyze whether the stock looks overbought (RSI > 70) or oversold (RSI < 30).

> ***Note:*** *The free tier has a 25 calls/day limit. This is plenty for personal monitoring but will need an upgrade ($49.99/month) if you're running frequent automated checks.*

Service 3: NewsAPI

What it does: Aggregates news headlines from 80,000+ sources worldwide. Filter by country, category, keyword, or source.

Get your key:

1. Go to newsapi.org
2. Register for a free account
3. Copy your API key (free tier: 100 requests/day)

Add to `.env`:

```
NEWSAPI_KEY=your-key-here
```

Test it: Ask your AI: "What are the top 5 business headlines right now?"

Your AI will pull live headlines from major news sources and present them with titles, sources, and brief descriptions.

Service 4: Supabase

What it does: Provides a cloud PostgreSQL database with a REST API. Store data, run queries, and build data-driven automations.

Get your key:

1. Go to supabase.com and create a free account
2. Create a new project (choose a region close to you)
3. Go to Settings → API

4. Copy the Project URL and the `anon` (public) key

Add to `.env`:

```
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_ANON_KEY=eyJ...your-key-here
```

Test it: Ask your AI: "Check if the Supabase database is connected"

Your AI will ping the Supabase API and confirm the connection. Later, you can create tables to store automation logs, trade records, contact lists, or any structured data.

Creating your first table: Ask your AI: "Create a Supabase table called 'automation_logs' with columns: id (auto), timestamp, task, result, status"

Service 5: GitHub

What it does: Access your code repositories, create issues, check commits, and manage projects through the GitHub API.

Get your key:

1. Go to github.com → Settings → Developer Settings → Personal Access Tokens → Tokens (classic)
2. Click "Generate new token (classic)"
3. Name it "OpenClaw" and select scopes: `repo`, `read:user`
4. Copy the token

Add to `.env`:

```
GITHUB_TOKEN=github_pat_your-token-here
```

Test it: Ask your AI: "List my GitHub repositories"

Verifying All Connections

Once all 5 services are configured, run a comprehensive test:

Send to your AI: "Test all API connections and report the status of each"

Your AI should report something like:

```
API Connection Status:
✅ Brave Search — Working (web search functional)
✅ Alpha Vantage — Working (RSI data retrieved)
✅ NewsAPI — Working (3 headlines fetched)
✅ Supabase — Working (database responding)
✅ GitHub — Working (repositories accessible)
```

What You've Connected

At the end of this chapter, you have:

✅ Web search capability (Brave) ✅ Financial market data (Alpha Vantage) ✅ Real-time news aggregation (NewsAPI) ✅ Cloud database storage (Supabase) ✅ Code repository access (GitHub)

Your AI now has hands. It can search, analyze markets, read news, store data, and manage code. Each of these capabilities can be combined and automated — which is exactly what we'll do after building the memory system.

Next Chapter: Memory Systems — Giving Your AI a Brain →

Chapter 6: Memory Systems

Giving Your AI a Brain

The Amnesia Problem

Here's the dirty secret of AI assistants: they forget everything.

Every time you start a new session with ChatGPT, Claude, or any AI tool, it has zero memory of previous conversations. You're starting from scratch every single time. That's fine for one-off questions, but it's fatal for automation.

An automation system needs to remember:

- What you discussed yesterday
- What projects are in progress
- What your preferences are
- What happened in previous sessions
- What tasks are pending

OpenClaw solves this with a file-based memory system. Your AI reads and writes memory files that persist across sessions. When it wakes up, it reads its memory. When something important happens, it writes it down.

How Memory Works

OpenClaw's memory system has three layers:

```
┌──────────────────────────────────┐
│           MEMORY.md              │ ← Curated long-term memory
│    (The important stuff, distilled)  │
├──────────────────────────────────┤
│      memory/YYYY-MM-DD.md        │ ← Daily logs
│    (Raw notes from each day)     │
├──────────────────────────────────┤
│         Session Context          │ ← Current conversation
```

```
│    (In-memory, resets each session)  │
└──────────────────────────────────────┘
```

Session Context is automatic — OpenClaw maintains context within a conversation. You don't need to configure anything.

Daily Logs are semi-automatic. Your AI writes to `memory/YYYY-MM-DD.md` during sessions. You can also write to them manually or ask your AI to record specific things.

Long-term Memory (MEMORY.md) is curated. This is the distilled essence of everything important. Your AI should periodically review daily logs and update MEMORY.md with the insights worth keeping permanently.

Setting Up the Memory Structure

Your workspace should already have:

```
~/.openclaw/workspace/
├── MEMORY.md           ← Long-term memory (create if missing)
├── memory/             ← Daily logs directory (create if missing)
│   └── .gitkeep
```

If the `memory/` directory doesn't exist, create it:

```
mkdir -p ~/.openclaw/workspace/memory
```

Configuring Memory Behavior

In your `AGENTS.md`, add memory instructions:

```
## Memory Protocol

### Every Session Start
1. Read MEMORY.md for long-term context
2. Read memory/YYYY-MM-DD.md for today's notes
3. Read yesterday's notes if today's file is empty

### During Sessions
- Write significant events to today's daily log
- Record decisions, outcomes, and lessons learned
- Note any tasks that were completed or created
- Log API results that might be useful later

### Memory Maintenance
```

```
- Periodically review recent daily logs
- Update MEMORY.md with important long-term information
- Remove outdated information from MEMORY.md
- Keep MEMORY.md concise and organized
```

What to Remember

Not everything needs to be stored. Here's a framework:

Always record:

- Decisions and the reasoning behind them
- Project milestones and status changes
- New preferences or requirements you mention
- Errors encountered and how they were resolved
- API configuration changes
- Important dates and deadlines

Skip:

- Casual conversation
- Information easily found online
- Temporary data that expires quickly
- Sensitive credentials (never store in memory files)

The Memory Lifecycle

Here's how memory flows through the system:

Day 1: You have a conversation about setting up a trading bot.

Your AI writes to `memory/2026-03-08.md`:

```
# 2026-03-08

## Trading Bot Discussion
- User wants to monitor AAPL, TSLA, NVDA
- RSI-based alerts when crossing 70 or 30
- Paper trading only for now
- Check every 2 hours during market hours
```

Day 7: Your AI reviews the week's daily logs and updates MEMORY.md:

```
## Active Projects
- **Trading Bot**: RSI monitoring for AAPL, TSLA, NVDA
  - Paper trading mode
  - 2-hour check interval during market hours
  - Status: Running since March 8
```

Day 30: Your AI has a rich context. When you say "How's the trading bot doing?", it knows exactly what you mean, what symbols you're tracking, and can pull the latest data immediately.

Testing Memory

Test 1: Short-term Memory

```
You: My project deadline is March 15th. Remember that.
AI: Noted — project deadline March 15th.

[Next message in same session]
You: When's my deadline?
AI: March 15th.
```

Test 2: Cross-session Memory

```
Session 1:
You: I prefer morning briefings at 8 AM Central time.
AI: [Records to memory file]

Session 2 (next day):
You: What time do I want my briefings?
AI: You prefer morning briefings at 8 AM Central time.
```

Test 3: Memory File Verification

```
You: Show me what's in today's memory file.
AI: [Reads and displays memory/2026-03-08.md]
```

Advanced: Memory Search

OpenClaw includes semantic memory search. When your AI needs to recall something, it searches across all memory files to find relevant context:

```
You: What did we decide about the email automation last week?
AI: [Searches memory files, finds the relevant entry, responds with context]
```

This means your AI doesn't need to read every memory file every session — it can search for specific topics efficiently.

What You've Built

At the end of this chapter, you have:

✅ Daily log system recording session events ✅ Long-term memory curation in MEMORY.md ✅ Memory reading on session start for context ✅ Cross-session continuity — your AI remembers ✅ Semantic search across memory files

Your AI now has a brain. It remembers past conversations, accumulates knowledge, and maintains context across sessions. This is the foundation that makes automation truly intelligent.

Next Chapter: Automation — Cron Jobs & Scheduled Tasks →

Chapter 7: Automation

Cron Jobs & Scheduled Tasks

From Reactive to Proactive

Up to now, your AI responds when you message it. That's reactive. The real power comes when your AI acts on its own — checking data, sending alerts, and executing tasks on a schedule without any input from you.

This is the layer that turns an assistant into a workforce.

Understanding Cron Jobs

A cron job is a scheduled task that executes automatically at specified times. OpenClaw's cron system supports three schedule types:

One-shot timers: "Run this task in 20 minutes"

```
{ "kind": "at", "at": "2026-03-08T20:30:00Z" }
```

Recurring intervals: "Run this task every 4 hours"

```
{ "kind": "every", "everyMs": 14400000 }
```

Cron expressions: "Run every weekday at 8 AM Central"

```
{ "kind": "cron", "expr": "0 8 * * 1-5", "tz": "America/Chicago" }
```

Your First Cron Job: Morning Briefing

Let's build the most useful automation first — a daily morning briefing that arrives in your inbox before you start your day.

Tell your AI:

```
Create a cron job that runs every morning at 8 AM Central time.
It should:
1. Check today's top 5 business headlines
2. Search for any news about AI automation
3. Summarize everything into a clean briefing
4. Send it to me via Telegram
```

Your AI will create the cron job with a schedule and task definition. The result: every morning at 8 AM, you receive a formatted news briefing in Telegram without lifting a finger.

Building Common Automations

Here are the most valuable cron jobs for personal use:

Weather Alert (Morning)

Schedule: Every day at 7 AM **Task:** Check the weather forecast. If rain is expected, alert me. Otherwise, don't bother.

This is a smart automation — it only contacts you when there's something worth knowing.

Weekly Review (Sunday Evening)

Schedule: Every Sunday at 8 PM **Task:** Summarize this week's memory files. What was accomplished? What's pending? What should I focus on next week?

Your AI reads the week's daily logs and produces a structured review. It's like having a personal chief of staff.

Market Check (Business Hours)

Schedule: Every 2 hours, 9 AM to 4 PM, weekdays **Task:** Check RSI indicators for my watchlist. Only alert me if any symbol crosses above 70 or below 30.

This is conditional automation — your AI checks, but only interrupts you when action might be needed.

Daily Memory Maintenance

Schedule: Every day at 11 PM **Task:** Review today's memory file. Extract any important information and update MEMORY.md. Clean up temporary notes.

This keeps your memory system healthy without manual intervention.

Cron Job Management

List all jobs:

```
What cron jobs are currently active?
```

Pause a job:

```
Pause the morning briefing cron job
```

Delete a job:

```
Remove the weather alert cron job
```

Run a job immediately:

```
Run the morning briefing now (don't wait for the schedule)
```

Heartbeat System

Beyond cron jobs, OpenClaw has a heartbeat system — periodic check-ins where your AI can proactively do useful work.

Create a `HEARTBEAT.md` file in your workspace:

```
# HEARTBEAT.md

## Checks to perform (rotate through these):
- [ ] Check for urgent news about topics I follow
- [ ] Review pending tasks in memory
- [ ] Check if any memory maintenance is needed
```

When the heartbeat fires (typically every 30-60 minutes), your AI reads HEARTBEAT.md and performs the listed checks. If something needs attention, it messages you. If not, it stays quiet.

The key principle: **your AI should reach out when it has something useful to say, and stay silent when it doesn't.** Nobody wants an AI that messages "nothing new!" every 30 minutes.

Smart Automation Principles

1. **Conditional alerts** — Don't alert on every check. Alert only when thresholds are crossed or anomalies detected.
2. **Batch operations** — Combine multiple checks into one cron job instead of creating many separate jobs.
3. **Quiet hours** — Configure your AI to suppress non-urgent alerts between 11 PM and 7 AM.
4. **Escalation** — Low-priority items get logged. Medium-priority items get a notification. High-priority items get immediate attention.
5. **Cost awareness** — Use cheap models (DeepSeek, Flash) for cron jobs. They run unattended and don't need premium reasoning.

Testing Your Automations

After creating a cron job, test it before waiting for the schedule:

```
Run the morning briefing cron job now
```

This executes the task immediately so you can verify the output format, content quality, and delivery.

Check if automations ran successfully:

```
Show me the results of the last cron job execution
```

What You've Automated

At the end of this chapter, you have:

✅ Morning briefing delivering news to your inbox daily ✅ Understanding of cron expressions and scheduling ✅ Conditional automation that only alerts when necessary ✅ Heartbeat system for periodic background work ✅ Smart automation principles for building more

This is where the compound effect kicks in. Every automation you add runs without ongoing effort. After a month, you'll have dozens of small automations doing useful work around the clock.

Next Chapter: Deployment — Going Live on Vercel →

Chapter 8: Deployment

Going Live on Vercel

Why Deploy?

Your AI system runs locally on your machine. That works great when your computer is on. But what about when you close your laptop? When you travel? When your machine restarts?

Deployment means your AI system's web presence — your blog, your landing page, your API endpoints — lives on the internet, accessible 24/7, regardless of what your local machine is doing.

In this chapter, we'll deploy a professional web presence using Vercel, the platform behind some of the internet's fastest websites.

> ***Note:*** *Your OpenClaw gateway still runs locally (or on a VPS). What we're deploying here is the web-facing portion: blog, landing pages, API endpoints for things like payment processing.*

Step 1: Create a Vercel Account

1. Go to vercel.com and sign up (free tier is generous)
2. Connect your GitHub account during signup
3. You'll get a free `.vercel.app` subdomain immediately

Step 2: Install the Vercel CLI

```
npm install -g vercel
```

Authenticate:

```
vercel login
```

Follow the prompts to log in via your browser.

Step 3: Build Your Project

If you have a web project (blog, landing page, dashboard), navigate to the project directory:

```
cd your-project-directory
vercel
```

Vercel automatically detects your framework (Astro, Next.js, etc.) and configures the build.

For your first deploy, accept the defaults:

- Link to existing project? No, create new
- Project name? Accept default or choose one
- Framework? Auto-detected
- Build settings? Accept defaults

Step 4: Deploy to Production

```
vercel --prod
```

Vercel builds your project and deploys it. You'll get a URL like:

```
https://your-project.vercel.app
```

That's it. Your site is live on the internet, with:

- Global CDN distribution
- Automatic SSL (HTTPS)
- Serverless functions for API endpoints
- Zero infrastructure management

Step 5: Connect a Custom Domain

If you have a custom domain:

1. In Vercel dashboard → your project → Settings → Domains
2. Type your domain name and click Add

3. Add the DNS records Vercel shows you to your domain registrar
4. Wait for DNS propagation (usually minutes with Cloudflare, up to 48 hours with others)

Step 6: Environment Variables

For features that need API keys (like Stripe checkout), add environment variables in Vercel:

1. Dashboard → Project → Settings → Environment Variables
2. Add each key-value pair
3. Select which environments need the variable (Production, Preview, Development)
4. Redeploy for changes to take effect

Step 7: Continuous Deployment

Connect your GitHub repository to Vercel for automatic deployments:

1. Dashboard → Project → Settings → Git
2. Connect your GitHub repository
3. Now every `git push` to `main` automatically triggers a new deployment

Your workflow becomes:

1. Make changes locally
2. `git add . && git commit -m "Update" && git push`
3. Vercel automatically builds and deploys
4. Live site updates in ~30 seconds

Performance Optimization

Vercel + Astro gives you exceptional performance out of the box:

- **Static generation** — Pages are pre-built HTML, served instantly from CDN
- **Zero JavaScript by default** — Only loads JS where you explicitly need it
- **Image optimization** — Automatic compression and format selection
- **Edge caching** — Content served from the nearest data center to each visitor

Target: 95+ Lighthouse score on all pages.

What You've Deployed

At the end of this chapter, you have:

✅ A live website accessible from anywhere in the world ✅ Automatic SSL/HTTPS security ✅ Global CDN for fast loading everywhere ✅ Continuous deployment from GitHub ✅ Environment variables for secure API integration ✅ A professional web presence for your AI automation brand

---

Congratulations!

You've built a complete AI automation system:

🏗️ **Infrastructure** — OpenClaw gateway running with workspace configured 📱 **Command Interface** — Telegram or Discord connected for mobile access 🧠 **AI Engine** — Multiple models configured with smart routing 🔌 **APIs** — 5 services connected (search, markets, news, database, code) 💾 **Memory** — Persistent context that survives across sessions ⚡ **Automation** — Cron jobs running tasks on schedule 🌐 **Deployment** — Live web presence on Vercel

This is your foundation. Everything you build from here compounds on what you've already set up. New APIs add new capabilities. New cron jobs add new automations. New memory entries make your AI smarter.

Want to go further? The Business Edition covers:

- Multi-engine routing with automatic cost optimization
- Full 14-API toolchain with reusable helper functions
- Trading bot framework with Alpaca integration
- Advanced memory systems with active context management
- Custom skill development
- Email automation with Resend
- Stripe payment integration

Visit erronatus.com to upgrade.

Appendices follow →

Appendix A: Troubleshooting & FAQ

Common Issues

"OpenClaw command not found"

Cause: npm global bin directory not in PATH **Fix (Windows):** Close PowerShell and reopen. If still failing: `npm config get prefix` — add that path + `/bin` to your system PATH. **Fix (macOS/Linux):** Run `export PATH="$(npm config get prefix)/bin:$PATH"` and add it to your `~/.bashrc` or `~/.zshrc`.

"Gateway fails to start"

Cause: Port already in use or configuration error **Fix:**

1. Check if another process uses the port: `netstat -an | findstr 18789` (Windows) or `lsof -i :18789` (macOS/Linux)
2. Change the port in `config.yaml`
3. Check logs: `openclaw gateway logs`

"AI doesn't respond on Telegram"

Cause: Bot token incorrect, gateway not running, or sender not authorized **Fix:**

1. Verify gateway is running: `openclaw status`
2. Check bot token in config.yaml matches BotFather's token exactly
3. Verify your Telegram user ID is in `allowedSenders`
4. Restart gateway: `openclaw gateway restart`

"API returns errors"

Cause: Invalid key, expired key, or rate limit exceeded **Fix:**

1. Verify the key in `.env` has no extra spaces or newlines

2. Test the key directly at the provider's website
3. Check if you've exceeded the free tier limits
4. Ensure the key has the required permissions/scopes

"Memory files aren't being created"

Cause: Directory doesn't exist or permissions issue **Fix:**

1. Create the directory: `mkdir -p ~/.openclaw/workspace/memory`
2. Check permissions: your user should own the workspace directory
3. Verify AGENTS.md has memory instructions

"Cron jobs don't fire"

Cause: Gateway not running, timezone mismatch, or job disabled **Fix:**

1. Verify gateway is running continuously
2. Check timezone in your cron expression matches your timezone
3. List jobs and verify they're enabled
4. Run the job manually to test

Frequently Asked Questions

Q: Can I run OpenClaw on a Raspberry Pi? A: Yes. OpenClaw runs anywhere Node.js runs. A Raspberry Pi 4 with 4GB RAM handles it well for personal use.

Q: How much does it cost per month? A: Infrastructure is free. AI model costs depend on usage — typically $30-150/month for active personal use. With smart engine routing, you can keep it under $50/month easily.

Q: Can I use multiple AI providers? A: Yes. OpenClaw supports switching between providers. Use OpenRouter for multi-provider access through a single key.

Q: Is my data private? A: Your data stays on your machine. OpenClaw doesn't send data to any Erronatus servers. The only external communication is between your gateway and the AI providers you configure.

Q: Can I use this for my business? A: The Personal Edition is licensed for personal use. For business use, upgrade to the Business Edition at erronatus.com.

Q: What happens if my computer shuts down? A: The gateway stops, but your configuration and memory persist on disk. When you restart the gateway, everything resumes. For 24/7 uptime, consider running on a VPS (covered in the Business Edition).

Appendix B: Glossary

Term	Definition
API	Application Programming Interface — a way for software to communicate with external services
API Key	A secret string that authenticates your access to an API
Cron Job	A scheduled task that runs automatically at specified times
Gateway	OpenClaw's core process that routes messages and manages AI interactions
LLM	Large Language Model — the AI models that generate responses (GPT, Claude, etc.)
Model	A specific AI system trained to process language (e.g., Claude Sonnet, GPT-4o)
OpenRouter	A service that provides access to multiple AI models through a single API key
Provider	A company that offers AI models (Anthropic, OpenAI, Google, etc.)
Session	A single conversation thread between you and your AI
Token	The unit AI models use to measure text — roughly 4 characters per token
Webhook	An HTTP callback that notifies your system when an event occurs
Workspace	The directory where OpenClaw stores configuration, memory, and project files

---

End of Personal Edition

For the complete system — multi-engine routing, full API toolchain, trading bots, email automation, and more — upgrade to the Business Edition at erronatus.com.

Chapter 9: Multi-Engine Routing

The Cost Optimization Playbook

Beyond Manual Model Switching

In Chapter 4, you learned to switch models manually with `/model flash` or `/model sonnet`. That works for interactive use. But for a business operation running dozens of automated tasks daily, manual switching is inefficient and error-prone.

Multi-engine routing automates model selection. You define rules. Your system picks the cheapest capable model for every task automatically.

The Routing Architecture

```
┌──────────────────────┐
│    Incoming Task     │
├──────────────────────┤
│    Route Classifier  │ ← What type of task is this?
├──────────────────────┤
│    Engine Selection  │ ← Which model handles this tier?
├──────────────────────┤
│    Budget Check      │ ← Are we within daily limits?
├──────────────────────┤
│    Execute & Log     │ ← Run task, record cost
└──────────────────────┘
```

Defining Your Engine Tiers

Create an engine routing configuration. Here's the production-tested setup we use:

Tier	Alias	Model	Cost/1M tokens	Use Case
Free	flash	Gemini 2.0 Flash Lite	$0.00	Status, weather, simple lookups

Budget	deepseek	DeepSeek V3.2	$0.27 input / $1.10 output	Parsing, summaries, cron jobs
Standard	sonnet	Claude Sonnet 4	$3.00 / $15.00	Research, coding, analysis
Premium	opus	Claude Opus 4	$15.00 / $75.00	Strategy, architecture, complex reasoning

The cost difference is massive. A task that costs $0.10 on Opus costs $0.001 on DeepSeek. That's 100x savings for tasks that don't need premium reasoning.

Configuring the Router

Create `~/.openclaw/engine-router.json`:

```
{
  "defaultEngine": "sonnet",
  "dailyBudget": 15.00,
  "warnThreshold": 0.80,
  "hardStopFallback": "flash",
  "tiers": {
    "free": {
      "model": "openrouter/google/gemini-2.0-flash-lite-001",
      "maxCostPerCall": 0,
      "tasks": ["status", "weather", "time", "simple_math", "health_check"]
    },
    "budget": {
      "model": "openrouter/deepseek/deepseek-v3.2",
      "maxCostPerCall": 0.01,
      "tasks": ["summarize", "parse", "cron", "monitor", "draft", "format"]
    },
    "standard": {
      "model": "anthropic/claude-sonnet-4-20250514",
      "maxCostPerCall": 0.10,
      "tasks": ["research", "code", "analyze", "write", "debug", "plan"]
    },
    "premium": {
      "model": "anthropic/claude-opus-4-6",
      "maxCostPerCall": 1.00,
      "tasks": ["strategy", "architecture", "financial_model", "complex_reasoning"]
    }
  },
  "fallbackChain": ["sonnet", "deepseek", "mini", "flash"]
}
```

Budget Management

The budget system prevents runaway costs:

- **Daily budget: $15.00** — Total spend cap per 24-hour period
- **Warning at 80% ($12.00)** — Your AI notifies you and starts preferring cheaper models
- **Hard stop at 100% ($15.00)** — All tasks fall back to the free tier
- **Fallback chain** — If the selected model fails, try the next one in the chain

Cost logging:

Every API call gets logged to `~/.openclaw/logs/cost-log.jsonl`:

```
{"timestamp":"2026-03-08T14:30:00Z","model":"sonnet","tokens_in":1500,"tokens_out":800,
{"timestamp":"2026-03-08T14:31:00Z","model":"deepseek","tokens_in":2000,"tokens_out":50
```

This gives you full visibility into where your money goes. After a week of logging, you'll know exactly which tasks consume the most budget and where to optimize.

Real-World Cost Analysis

Here's a typical day for a business user with smart routing:

Time	Task	Model	Cost
8:00	Morning briefing (cron)	deepseek	$0.003
8:05	Weather check	flash	$0.000
9:00	Code review	sonnet	$0.045
10:00	Market scan (cron)	deepseek	$0.002
11:00	Research task	sonnet	$0.060
12:00	Market scan (cron)	deepseek	$0.002
13:00	Email draft	deepseek	$0.004
14:00	Market scan (cron)	deepseek	$0.002
15:00	Architecture decision	opus	$0.350
16:00	Market scan (cron)	deepseek	$0.002
17:00	Blog post writing	sonnet	$0.080

21:00	Evening review (cron)	deepseek	$0.005
23:00	Memory maintenance (cron)	flash	$0.000
Total			**$0.555**

That's 55 cents for a full day of AI operations across 13 tasks. Even on heavy days with multiple Opus calls, you'll rarely exceed $5-10.

Compare that to:

- ChatGPT Pro subscription: $200/month (no automation)
- Claude Pro subscription: $20/month (no automation)
- Virtual assistant: $15-30/hour
- Custom AI SaaS: $500-2000/month

Optimization Strategies

1. Cron jobs always use budget tier Cron jobs run unattended. They don't need premium reasoning — they need to check data, format results, and deliver. DeepSeek handles this perfectly at 1/10th the cost.

2. Reserve premium for decision-making The only tasks that justify Opus pricing are decisions with significant consequences: trading strategies, architectural choices, financial projections, legal analysis.

3. Use context to reduce tokens Longer prompts cost more. Keep your system prompts concise. Use memory references instead of pasting full context into every message.

4. Batch similar tasks Instead of 5 separate market checks, run one cron job that checks all 5 symbols and reports together.

5. Monitor and adjust weekly Review your cost log every week. Identify any tasks running on more expensive models than necessary and adjust routing.

What You've Optimized

At the end of this chapter, you have:

✅ Automatic model routing based on task complexity ✅ Daily budget with warning thresholds and hard stops ✅ Cost logging for full spending visibility ✅ Fallback chain for reliability ✅ Production-tested routing configuration

Your AI now makes smart economic decisions about which brain to use for each task. The savings compound daily.

Next Chapter: The Full API Toolchain →

Chapter 10: The Full API Toolchain

14 Services, One Script

From Individual APIs to a Unified Toolchain

In Chapter 5, you connected 5 APIs individually. That's fine for personal use. For business operations, you need a systematic approach — a single, reusable script that wraps every service into clean function calls.

This chapter gives you that script. We call it `api-tools.js` — a production-tested toolchain that we use daily.

The Architecture

```
┌──────────────────────────────────────┐
│           api-tools.js               │
│                                      │
│  ┌─────────┐  ┌─────────┐  ┌──────┐ │
│  │ Trading │  │  Data   │  │ Infra│ │
│  │         │  │         │  │      │ │
│  │ Alpaca  │  │ AlphaV  │  │Vercel│ │
│  │         │  │ NewsAPI │  │ CF   │ │
│  │         │  │ Supabase│  │GitHub│ │
│  └─────────┘  └─────────┘  └──────┘ │
│                                      │
│  ┌─────────┐  ┌─────────┐           │
│  │  Comms  │  │ Payments│           │
│  │         │  │         │           │
│  │ Resend  │  │ Stripe  │           │
│  │         │  │         │           │
│  └─────────┘  └─────────┘           │
│                                      │
│  All credentials loaded from .env    │
│  All functions export clean interfaces│
└──────────────────────────────────────┘
```

Setting Up All 14 Services

Here's every API key you need, organized by category:

Trading & Finance:

```
ALPACA_API_KEY=your-paper-trading-key
ALPACA_SECRET_KEY=your-paper-secret-key
ALPACA_BASE_URL=https://paper-api.alpaca.markets
ALPHA_VANTAGE_KEY=your-key
```

Social & Communication:

```
X_API_KEY=your-twitter-api-key
X_API_SECRET=your-twitter-secret
X_ACCESS_TOKEN=your-access-token
X_ACCESS_SECRET=your-access-secret
X_BEARER_TOKEN=your-bearer-token
RESEND_API_KEY=re_your-resend-key
```

Development & Deployment:

```
GITHUB_TOKEN=github_pat_your-token
VERCEL_TOKEN=your-vercel-token
CLOUDFLARE_API_TOKEN=your-cf-token
CLOUDFLARE_ACCOUNT_ID=your-account-id
```

Database & Services:

```
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_ANON_KEY=your-anon-key
NEWSAPI_KEY=your-newsapi-key
```

Payments:

```
STRIPE_SECRET_KEY=sk_test_your-key
STRIPE_PUBLISHABLE_KEY=pk_test_your-key
```

Search & AI:

```
OPENROUTER_API_KEY=sk-or-v1-your-key
BRAVE_SEARCH_KEY=your-brave-key
```

The api-tools.js Script

Create `~/.openclaw/workspace/scripts/api-tools.js`:

This ES module script:

1. Loads all credentials from `~/.openclaw/.env` automatically
2. Validates that required keys exist before making calls
3. Exports clean async functions for each service
4. Handles errors gracefully with descriptive messages

Available Functions:

```
// Trading
alpaca_get_price(symbol)        // Latest stock price
alphavantage_rsi(symbol)        // RSI technical indicator

// News & Data
newsapi_headlines(count)        // Top business headlines

// Infrastructure
github_list_repos()             // List GitHub repositories
supabase_query(table, select)   // Query Supabase database
cloudflare_verify()             // Verify Cloudflare token
vercel_user()                   // Get Vercel account info

// Payments
stripe_balance()                // Check Stripe balance

// Email
resend_list_keys()              // List Resend API keys
```

Usage in your AI's context:

Your AI can call these functions directly:

```
You: What's the current price of AAPL?
AI: [Calls alpaca_get_price('AAPL')]
    AAPL is trading at $178.52 as of 2:30 PM ET.

You: Check my Stripe balance
AI: [Calls stripe_balance()]
```

```
    Stripe balance: $1,247.00 available, $89.00 pending.

You: Get the latest RSI for NVDA
AI: [Calls alphavantage_rsi('NVDA')]
    NVDA RSI (14-period, 5min): 45.2 — neutral territory.
```

Building the Script

Here's the complete implementation pattern. Each function follows the same structure:

```javascript
import { readFileSync } from 'fs';
import { join } from 'path';

// Load .env
const envPath = join(process.env.HOME || process.env.USERPROFILE, '.openclaw', '.env');
const envVars = {};

const envContent = readFileSync(envPath, 'utf8');
envContent.split('\n').forEach(line => {
  line = line.trim();
  if (line && !line.startsWith('#')) {
    const eqIndex = line.indexOf('=');
    if (eqIndex > 0) {
      envVars[line.slice(0, eqIndex)] = line.slice(eqIndex + 1);
    }
  }
});

function requireKey(key, service) {
  if (!envVars[key]) throw new Error(`Missing ${service} key: ${key}`);
  return envVars[key];
}

// Example: Alpaca
export async function alpaca_get_price(symbol) {
  const API_KEY = requireKey('ALPACA_API_KEY', 'Alpaca');
  const SECRET = requireKey('ALPACA_SECRET_KEY', 'Alpaca');

  const response = await fetch(
    `https://data.alpaca.markets/v2/stocks/${symbol}/trades/latest`,
    {
      headers: {
        'APCA-API-KEY-ID': API_KEY,
        'APCA-API-SECRET-KEY': SECRET
      }
    }
  );

  const data = await response.json();
```

```
  return { symbol, price: data.trade.p, timestamp: data.trade.t };
}
```

The full script includes all 9+ functions with error handling, fallback logic, and clean return types. The complete, tested source code is included in the downloadable package.

Testing the Toolchain

Run a comprehensive verification of all services:

```
You: Test all API connections and report status
AI: Running API verification...

✅ Alpaca Trading — Paper account active
✅ Alpha Vantage — AAPL RSI retrieved (42.8)
✅ NewsAPI — 5 headlines fetched
✅ GitHub — @YourUsername, 12 repos
✅ Supabase — Database responding
✅ Stripe — Balance API working ($0.00)
✅ Cloudflare — Token verified, active
✅ Vercel — Account accessible
✅ Resend — 1 API key found

9/9 services operational.
```

What You've Built

At the end of this chapter, you have:

✅ A unified API toolchain script with 9 helper functions ✅ All 14 services configured and authenticated ✅ Clean function interfaces your AI calls directly ✅ Automatic credential loading from .env ✅ Error handling and descriptive failure messages ✅ A verification routine to test everything at once

This toolchain is the backbone of every automation you'll build. Trading bots call `alpaca_get_price()`. Morning briefings call `newsapi_headlines()`. Email automations call Resend. Everything routes through one tested, reliable script.

Next Chapter: Trading Automation →

Chapter 11: Trading Automation

Alpaca, RSI, and Paper Trading Bots

> ⚠️ ***Disclaimer:*** *This chapter covers paper trading (simulated trades with fake money). Nothing in this guide constitutes financial advice. Trading real money involves significant risk of loss. Always paper trade extensively before considering live trading.*

Why AI + Trading

Traditional algorithmic trading uses rigid if-then rules. When market conditions change, rigid bots break.

AI-powered trading automation is different:

- **Contextualizes signals** with news, sector trends, and historical patterns
- **Adapts analysis** to current market conditions
- **Explains reasoning** in plain English before execution
- **Remembers** what worked and what didn't over weeks and months

You're not building a trading bot. You're building a trading *analyst* that never sleeps.

Setting Up Alpaca Paper Trading

Alpaca provides free paper trading with the same API as live trading.

1. Go to alpaca.markets and create an account
2. Switch to "Paper Trading" in the dashboard
3. Go to API Keys → Generate New Key
4. Copy both the API Key ID and Secret Key

Add to `.env`:

```
ALPACA_API_KEY=your-paper-key
ALPACA_SECRET_KEY=your-paper-secret
ALPACA_BASE_URL=https://paper-api.alpaca.markets
```

Verify the connection:

```
You: Check my Alpaca paper trading account status
AI: Paper account active. Buying power: $100,000.00. No open positions.
```

Building the Watchlist Monitor

Define 5-10 symbols you want to track:

```
You: Set up a watchlist monitor for AAPL, TSLA, NVDA, AMZN, GOOGL.
     Check RSI every 2 hours during market hours.
     Alert me only when RSI crosses above 70 or below 30.
```

Your AI creates a cron job:

- **Schedule:** Every 2 hours, 9 AM - 4 PM ET, weekdays
- **Task:** For each symbol: pull current price + RSI → analyze → alert if threshold crossed → log everything

The Three-Tier Alert System

🟢 **Info (Logged, not sent)** Regular scans with no signals. Written to daily memory for pattern analysis.

```
[Market Scan 10:00] AAPL RSI 52.3 (neutral) | TSLA RSI 61.8 (neutral) |
NVDA RSI 44.1 (neutral) | All clear.
```

🟡 **Watch (Telegram notification)** A signal is developing. RSI approaching thresholds.

```
⚠️ NVDA RSI approaching oversold: 33.2 and falling.
50-day SMA support at $112. Watching for entry below 30.
```

🔴 **Action (Priority alert with recommendation)** Signal confirmed with specific trade parameters.

```
🔴 NVDA RSI crossed below 30 (29.4)
📊 Analysis: Oversold on 5min RSI, price at 50-day SMA support ($112)
💡 Recommendation: Paper buy 45 shares at market (~$5,040)
```

```
🛑 Stop: $108 (-3.6%)
🎯 Target: $120 (+7.1%)
⚖️ Risk: $180 (0.18% of account)

Reply "execute" to place the paper trade.
```

AI-Powered Position Sizing

Your AI calculates optimal position size for every trade:

```
Position Size = (Account Equity × Risk %) / (Entry Price - Stop Price)
```

But it adds judgment:

- **Sector concentration:** Already holding 3 tech stocks? Reduce size.
- **Volatility adjustment:** High ATR? Wider stop, smaller position.
- **Correlation check:** Correlated positions share risk — account for it.
- **Account heat:** Total portfolio risk shouldn't exceed 6%.

Example:

```
Account: $100,000
Risk per trade: 1% = $1,000
Entry: $112.00
Stop: $108.00
Stop distance: $4.00

Position size: $1,000 / $4.00 = 250 shares ($28,000)

Adjustments:
- Already holding GOOGL (+$15,000 tech exposure) → Reduce to 150 shares
- ATR elevated (1.5x normal) → Reduce to 120 shares
- Final: 120 shares × $112 = $13,440 (13.4% of account)
```

Executing Paper Trades

When you confirm a trade:

```
You: Execute

AI: Placing paper trade...
    ✅ BUY 120 NVDA @ $112.03 (market order filled)
    📋 Order ID: abc123
    🛑 Stop-loss set: $108.00
    📊 Position: 120 shares, cost basis $13,443.60
    ⚖️ Portfolio risk: 2.3% (within limits)
```

```
    Monitoring for exit signals. Will alert at stop ($108) or target ($120).
```

Performance Tracking

Every trade gets logged with full context:

```
{
  "date": "2026-03-08",
  "symbol": "NVDA",
  "side": "buy",
  "shares": 120,
  "entry": 112.03,
  "stop": 108.00,
  "target": 120.00,
  "reasoning": "RSI oversold (29.4) at 50-day SMA support",
  "outcome": "pending"
}
```

After closing, your AI updates:

```
{
  "exit": 118.50,
  "pnl": "+$776.40 (+5.8%)",
  "days_held": 4,
  "outcome": "win",
  "lesson": "RSI < 30 at major support = high probability bounce"
}
```

Monthly Performance Report

Your AI generates automated performance analysis:

```
March 2026 Paper Trading Summary
────────────────────────────────
Total trades: 23
Win rate: 61% (14W / 9L)
Average gain: +2.3%
Average loss: -1.1%
Profit factor: 1.88
Max drawdown: -4.2%

Best trade: NVDA +8.7% (RSI < 30 bounce)
Worst trade: AMZN -3.2% (earnings miss override)

Pattern Analysis:
```

```
  • RSI < 30 at support: 78% win rate (7/9)
  • RSI > 70 shorts: 42% win rate (3/7) — underperforming
  • Recommendation: Increase allocation to oversold bounces,
    reduce or eliminate overbought short signals.
```

Risk Management Automation

Configure automated risk rules:

1. **Max position size:** 5% of account per trade
2. **Max portfolio heat:** 6% total open risk
3. **Sector concentration:** No more than 40% in one sector
4. **Daily loss limit:** -3% → suspend new entries, alert
5. **Trailing stops:** Move stops to breakeven after +2% gain

These rules execute automatically. Your AI doesn't get emotional after losses or greedy after wins. It follows the system.

What You've Built

✅ Paper trading bot with Alpaca integration ✅ RSI-based watchlist monitoring on cron ✅ Three-tier alert system (Info/Watch/Action) ✅ AI-powered position sizing with risk adjustments ✅ Automated trade execution via Telegram command ✅ Performance logging and monthly analysis ✅ Risk management automation with hard limits

---

Next Chapter: Advanced Memory Systems →

Chapter 12: Advanced Memory

Context Management at Scale

Beyond Simple Memory Files

The basic memory system from Chapter 6 works for personal use. But when you're running a business operation — multiple projects, trading positions, client work, content schedules — you need structured context management.

This chapter introduces three advanced memory systems:

1. **Active Context** — What's happening right now
2. **Task Queue** — What needs to happen next
3. **Credential Vault** — Secure reference index for all your keys

System 1: Active Context

Create `~/.openclaw/memory/active-context.json`:

```
{
  "lastSession": {
    "date": "2026-03-08",
    "summary": "Completed API toolchain setup. All 9/9 services verified.",
    "pendingFollowUp": ["Create trading cron job", "Test email automation"]
  },
  "activeProjects": [
    {
      "name": "erronatus-website",
      "status": "deployed",
      "lastUpdate": "2026-03-08",
      "nextAction": "Add Stripe products, connect custom domain"
    },
    {
      "name": "trading-bot",
      "status": "in-progress",
      "lastUpdate": "2026-03-08",
```

```
      "nextAction": "Set up RSI monitoring cron job"
    }
  ],
  "openPositions": [],
  "dailyBudgetUsed": 2.35,
  "importantDates": [
    { "date": "2026-03-15", "event": "Project deadline" }
  ]
}
```

Your AI reads this on session start to instantly know:

- What was the last thing you worked on
- What projects are active and their status
- What follow-up items are pending
- What positions are open
- How much budget has been used today

System 2: Task Queue

Create `~/.openclaw/memory/tasks/task-queue.json`:

```
{
  "queue": [
    {
      "id": "task-001",
      "priority": "high",
      "title": "Set up RSI monitoring cron for watchlist",
      "description": "Create cron job: every 2h, market hours, check AAPL/TSLA/NVDA/AMZ
      "created": "2026-03-08",
      "status": "pending",
      "assignedTo": "cron"
    },
    {
      "id": "task-002",
      "priority": "medium",
      "title": "Create Stripe products for Blueprint",
      "description": "Personal ($47), Business ($97), Enterprise ($299) — one-time paym
      "created": "2026-03-08",
      "status": "pending",
      "assignedTo": "manual"
    }
  ],
  "completed": [
    {
      "id": "task-000",
      "title": "Verify all API connections",
      "completed": "2026-03-08",
```

```
      "result": "9/9 pass"
    }
  ]
}
```

Your AI maintains this queue:

- New tasks get added when you mention them
- Completed tasks move to the `completed` array
- Priority determines what gets suggested first
- Your AI can proactively say: "You have 2 high-priority tasks pending. Want to tackle the RSI monitoring setup?"

System 3: Credential Vault Index

Create `~/.openclaw/memory/credentials/vault.json`:

```
{
  "credentials": [
    {
      "service": "Alpaca",
      "type": "paper-trading",
      "envKeys": ["ALPACA_API_KEY", "ALPACA_SECRET_KEY"],
      "status": "active",
      "lastVerified": "2026-03-08",
      "notes": "Paper account. $100k buying power."
    },
    {
      "service": "Stripe",
      "type": "payment-processing",
      "envKeys": ["STRIPE_SECRET_KEY", "STRIPE_PUBLISHABLE_KEY"],
      "status": "active",
      "lastVerified": "2026-03-08",
      "notes": "Live keys. Balance: $0.00"
    }
  ],
  "totalCredentials": 22,
  "lastFullAudit": "2026-03-08"
}
```

This is a reference index — it doesn't store the actual keys (those stay in `.env`). It tracks what credentials you have, their status, and when they were last verified. Your AI can reference this to know what services are available without reading the `.env` file directly.

Boot File: Session Initialization

Create `~/.openclaw/boot.md` — a comprehensive context file loaded on every session start:

```
# Boot Context

## Who I Am
[AI name and role]

## Engine Routing Rules
[Model tiers and routing logic]

## API Toolchain
[Available functions and their locations]

## Active Context
[Current project statuses and priorities]

## Key File Locations
[Paths to all important files]

## Standing Instructions
[Rules that always apply]
```

This single file gives your AI complete operational awareness within the first few seconds of every session. No warm-up period. No asking "what were we working on?" It knows.

Memory Maintenance Automation

Create a daily cron job for memory maintenance:

Schedule: Every day at 11 PM **Task:**

1. Read today's daily log
2. Extract significant events, decisions, and outcomes
3. Update active-context.json with current project states
4. Move completed tasks from queue to completed
5. Update MEMORY.md if any long-term insights emerged
6. Verify no sensitive data leaked into memory files

This runs automatically. Your memory system stays clean, organized, and current without manual intervention.

Project-Specific Memory

For larger projects, create dedicated memory directories:

```
~/.openclaw/memory/
├── projects/
│   ├── index.json              ← Project registry
│   ├── erronatus-website/
│   │   ├── decisions.md        ← Key decisions and rationale
│   │   ├── architecture.md     ← System design notes
│   │   └── changelog.md        ← What changed and when
│   └── trading-bot/
│       ├── strategy.md         ← Trading strategy documentation
│       ├── performance.md      ← Performance tracking
│       └── rules.md            ← Risk management rules
```

Your AI can navigate this structure to find relevant context for any project without loading everything into memory at once.

What You've Built

✅ Active context system for instant session awareness ✅ Task queue with priority management ✅ Credential vault index for service tracking ✅ Boot file for comprehensive session initialization ✅ Automated memory maintenance via cron ✅ Project-specific memory directories ✅ A memory system that scales with your operation

---

Next Chapter: Custom Skill Development →

Chapter 13: Custom Skill Development

Teaching Your AI New Tricks

What Are Skills?

Skills are packaged capabilities you can install into OpenClaw. Each skill includes a `SKILL.md` file that teaches your AI how to perform a specific type of task — weather lookups, deep research, document analysis, trading analysis, and more.

Think of skills as specialized training modules. Your AI reads the SKILL.md when a matching task is detected and follows its instructions precisely.

Anatomy of a Skill

Every skill lives in a directory with this structure:

```
~/.openclaw/workspace/skills/your-skill/
├── SKILL.md              ← Instructions your AI follows
├── scripts/              ← Helper scripts (optional)
│   └── helper.js
├── templates/            ← Output templates (optional)
│   └── report.md
└── README.md             ← Human-readable documentation
```

The `SKILL.md` is the critical file. It defines:

- **When** the skill should activate (trigger conditions)
- **What** the skill does (step-by-step process)
- **How** to format output (templates and standards)
- **Where** to find resources (scripts, APIs, reference data)

Building Your First Custom Skill

Let's build a "Client Report" skill that generates weekly performance reports for a business:

Create the directory:

```
mkdir -p ~/.openclaw/workspace/skills/client-report
```

Create `SKILL.md`:

```
# Client Report Skill

## Trigger
Activate when user asks to generate a client report, weekly report,
or performance summary.

## Process
1. Read the client configuration from memory/clients/{client_name}.json
2. Pull relevant metrics from the configured data sources
3. Compare current period to previous period
4. Identify trends, anomalies, and opportunities
5. Generate a formatted report using the template

## Data Sources
- Supabase: Query the metrics table for the client
- Google Analytics: Via web search for public metrics
- Custom APIs: As specified in client configuration

## Output Format
Use the template in templates/weekly-report.md
Include:
- Executive summary (3-4 sentences)
- Key metrics with trend arrows (↑↓→)
- Charts described in text format
- Recommendations (2-3 actionable items)
- Next steps

## Quality Standards
- All numbers must cite their source
- Percentages include comparison period
- Recommendations must be specific and actionable
- Report length: 500-1000 words
```

Create the template:

```
# Weekly Performance Report
**Client:** {{client_name}}
```

```
**Period:** {{start_date}} — {{end_date}}
**Generated:** {{generated_date}}

---

## Executive Summary
{{summary}}

## Key Metrics
| Metric | Current | Previous | Change |
|--------|---------|----------|--------|
{{#each metrics}}
| {{name}} | {{current}} | {{previous}} | {{change}} |
{{/each}}

## Analysis
{{analysis}}

## Recommendations
{{#each recommendations}}
{{@index}}. **{{title}}**: {{description}}
{{/each}}

## Next Steps
{{next_steps}}
```

Skill Discovery

OpenClaw automatically detects skills in your `skills/` directory. When you ask your AI to do something that matches a skill's trigger conditions, it loads and follows that skill's instructions.

You can also install pre-built skills from ClawHub:

```
clawhub search trading
clawhub install trading
clawhub install deep-research-pro
```

Installing Pre-Built Skills

Several powerful skills are available:

Skill	Purpose
`trading`	Technical analysis, chart patterns, risk management
`deep-research-pro`	Multi-source web research with cited reports

`market-research`	Market sizing, competitor analysis, opportunity validation
`weather`	Current weather and forecasts
`document-pro`	PDF, DOCX, PPTX processing and extraction
`document-summary`	Technical document summarization
`code`	Coding workflow with planning and testing

Install with:

```
clawhub install skill-name
```

Advanced Skill Patterns

Chained Skills: One skill's output feeds into another:

```
Research Skill → generates findings →
Analysis Skill → produces recommendations →
Report Skill → formats deliverable
```

Parameterized Skills: Accept configuration for different use cases:

```
## Parameters
- depth: quick | standard | deep
- format: summary | detailed | executive
- audience: technical | business | general
```

Scheduled Skills: Trigger via cron instead of user message:

```
Cron job → activates skill → generates output → delivers to channel
```

What You've Built

✅ Understanding of skill architecture and SKILL.md format ✅ A custom Client Report skill with templates ✅ Skill installation from ClawHub marketplace ✅ Advanced patterns: chaining, parameters, scheduled activation ✅ A framework for creating unlimited custom capabilities

Next Chapter: Email Automation →

Chapter 14: Email Automation

Resend Integration & Workflows

Why Email Automation

Email remains the most reliable channel for business communication. With Resend integration, your AI can:

- Send transactional emails (receipts, confirmations, notifications)
- Deliver scheduled reports to clients or team members
- Process incoming email summaries and draft responses
- Manage newsletter distribution
- Trigger email sequences based on events

Setting Up Resend

1. Go to resend.com and create an account
2. Verify your sending domain (requires DNS records)
3. Generate an API key
4. Add to `.env`:

```
RESEND_API_KEY=re_your-key-here
EMAIL_FROM=hello@yourdomain.com
```

Domain Verification

For email deliverability, verify your domain in Resend:

1. Go to Resend dashboard → Domains → Add Domain
2. Enter your domain (e.g., erronatus.com)

3. Add the DNS records Resend provides:
   - **SPF record** — Authorizes Resend to send from your domain
   - **DKIM record** — Cryptographically signs your emails
   - **DMARC record** — Policy for handling authentication failures

Add these records in your DNS provider (Cloudflare, Namecheap, etc.). Verification usually takes a few minutes.

Sending Your First Email

Test the integration:

```
You: Send a test email to me@example.com with subject "Test" and body "Email automation is w
AI: ✅ Email sent successfully.
    To: me@example.com
    Subject: Test
    Status: Delivered
```

Email Automation Workflows

1. Daily Report Delivery

Schedule a cron job to email you a daily summary:

```
Create a cron job: Every day at 6 PM, compile a summary of
today's activities, completed tasks, and open items.
Email the report to me at jackson@erronatus.com.
```

Your AI generates the report and sends it via Resend — no manual effort.

2. Client Update Emails

After generating a client report (Chapter 13's skill), automatically email it:

```
Generate the weekly report for Client X and email it to
client@company.com with the subject "Weekly Performance Report — March 8"
```

3. Alert Escalation to Email

For critical alerts that need more than a Telegram notification:

```
If a trading alert reaches 🔴 Action level, also send an email
with the full analysis to my business email.
```

4. Welcome Email Sequences

When someone purchases your product (via Stripe webhook), trigger a welcome sequence:

- **Immediately:** Purchase confirmation + download link
- **Day 1:** Getting started tips
- **Day 3:** "How's it going?" check-in with support link
- **Day 7:** Advanced features highlight
- **Day 14:** Request for review/testimonial

Email Templates

Create reusable HTML email templates that your AI populates with dynamic content:

```html
<!DOCTYPE html>
<html>
<body style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto;">
  <div style="background: #0a0a0a; padding: 40px; border-radius: 12px;">
    <img src="https://erronatus.com/logo.png" width="120" />
    <h1 style="color: #fff; margin-top: 24px;">{{title}}</h1>
    <p style="color: #a0a0a0; line-height: 1.6;">{{body}}</p>
    <a href="{{cta_url}}" style="display: inline-block; padding: 12px 24px;
       background: linear-gradient(135deg, #3b82f6, #8b5cf6); color: #fff;
       border-radius: 999px; text-decoration: none; margin-top: 16px;">
      {{cta_text}}
    </a>
  </div>
</body>
</html>
```

Store templates in `~/.openclaw/workspace/templates/emails/` and reference them in automations.

Deliverability Best Practices

1. **Always use verified domains** — Never send from gmail.com or yahoo.com
2. **Include unsubscribe links** — Required by law (CAN-SPAM, GDPR)
3. **Keep bounce rates low** — Remove invalid addresses promptly
4. **Warm up new domains** — Start with low volume, increase gradually
5. **Monitor reputation** — Check Resend analytics for delivery rates

What You've Built

✅ Resend email API integrated and verified ✅ Domain authentication (SPF, DKIM, DMARC) ✅ Automated report delivery via cron ✅ Alert escalation to email ✅ HTML email templates for professional communications ✅ Welcome sequence framework for product purchases

---

Next Chapter: Payment Integration →

Chapter 15: Payment Integration

Stripe Checkout & Revenue Systems

Building a Revenue Engine

Every business needs a way to accept money. Stripe is the industry standard — used by Amazon, Google, Shopify, and millions of businesses. With Stripe integration, your AI-powered system can:

- Accept one-time payments for digital products
- Process subscriptions for recurring revenue
- Handle refunds and disputes
- Track revenue analytics
- Trigger fulfillment workflows on successful payment

Stripe Account Setup

1. Go to stripe.com and create an account
2. Complete identity verification (required for live payments)
3. Navigate to Developers → API Keys
4. Copy your keys:

```
# Test mode (for development)
STRIPE_SECRET_KEY=sk_test_your-key
STRIPE_PUBLISHABLE_KEY=pk_test_your-key

# Live mode (for production — switch when ready)
# STRIPE_SECRET_KEY=sk_live_your-key
# STRIPE_PUBLISHABLE_KEY=pk_live_your-key
```

> ***Important:*** *Start with test keys. Test everything thoroughly. Switch to live keys only when your entire flow works perfectly.*

Creating Products and Prices

In the Stripe Dashboard → Products → Add Product:

Product 1: Personal Edition

- Name: "The Erronatus Blueprint — Personal Edition"
- Description: "Complete guide to AI automation with OpenClaw"
- Price: $47.00 (one-time)
- Copy the Price ID (e.g., `price_1ABC123`)

Product 2: Business Edition

- Name: "The Erronatus Blueprint — Business Edition"
- Price: $97.00 (one-time)
- Copy the Price ID

Product 3: Enterprise Edition

- Name: "The Erronatus Blueprint — Enterprise Edition"
- Price: $299.00 (one-time)
- Copy the Price ID

Building the Checkout Flow

The checkout process:

```
User clicks "Buy" → Your server creates a Stripe Checkout Session →
User redirects to Stripe's hosted payment page → User pays →
Stripe redirects to your success page → Webhook triggers fulfillment
```

Server endpoint (`/api/checkout`):

```
// Creates a Stripe Checkout Session
export async function POST({ request }) {
  const stripe = new Stripe(process.env.STRIPE_SECRET_KEY);
  const { priceId, email, edition } = await request.json();

  const session = await stripe.checkout.sessions.create({
    line_items: [{ price: priceId, quantity: 1 }],
    customer_email: email,
    mode: 'payment',
    success_url: 'https://yourdomain.com/thank-you?session_id={CHECKOUT_SESSION_ID}',
    cancel_url: 'https://yourdomain.com/#pricing',
    metadata: { edition },
```

```
    allow_promotion_codes: true,
  });

  return Response.json({ url: session.url });
}
```

Webhook-Powered Fulfillment

When a payment succeeds, Stripe sends a webhook to your server. This triggers automatic fulfillment:

1. **Set up webhook endpoint** in Stripe Dashboard → Webhooks → Add Endpoint
2. **URL:** `https://yourdomain.com/api/webhook`
3. **Events:** Select `checkout.session.completed`
4. **Copy the webhook signing secret** to your `.env`

Webhook handler:

```
export async function POST({ request }) {
  const stripe = new Stripe(process.env.STRIPE_SECRET_KEY);
  const signature = request.headers.get('stripe-signature');
  const body = await request.text();

  const event = stripe.webhooks.constructEvent(
    body,
    signature,
    process.env.STRIPE_WEBHOOK_SECRET
  );

  if (event.type === 'checkout.session.completed') {
    const session = event.data.object;
    const email = session.customer_email;
    const edition = session.metadata.edition;

    // Send the product via email
    await sendProductEmail(email, edition);

    // Log the sale
    await logSale(session);
  }

  return new Response('OK', { status: 200 });
}
```

Automated Product Delivery

When payment is confirmed, your system:

1. **Generates a secure download link** (signed URL, expires in 72 hours)
2. **Sends delivery email** via Resend with the download link
3. **Logs the sale** to Supabase for analytics
4. **Triggers welcome sequence** (Day 1, 3, 7, 14 emails)
5. **Updates your AI's memory** with the sale event

The buyer receives their product within seconds of payment. Fully automated.

Testing with Stripe Test Mode

Stripe provides test card numbers:

Card Number	Result
`4242 4242 4242 4242`	Successful payment
`4000 0000 0000 0002`	Declined
`4000 0000 0000 3220`	Requires 3D Secure

Use any future expiration date, any 3-digit CVC, and any ZIP code.

Test the full flow:

1. Click "Buy" on your website
2. Enter the test card number
3. Complete checkout
4. Verify redirect to success page
5. Check your email for the delivery
6. Verify the sale logged in Supabase

Revenue Analytics

Track your business metrics:

```
You: What are my sales numbers this month?
AI: [Queries Supabase sales table]

March 2026 Revenue Report:
──────────────────────────
Total sales: 47
Revenue: $3,891.00
```

```
Average order: $82.79

By edition:
- Personal ($47): 28 sales — $1,316
- Business ($97): 15 sales — $1,455
- Enterprise ($299): 4 sales — $1,196

Conversion rate: 3.2% (47 sales / 1,469 visitors)
```

What You've Built

✅ Stripe account configured with products and prices ✅ Checkout flow with hosted payment page ✅ Webhook-powered automatic fulfillment ✅ Secure download link generation ✅ Email delivery via Resend on purchase ✅ Sale logging to Supabase ✅ Revenue analytics accessible through your AI

Next Chapter: 24/7 Operations →

Chapter 16: 24/7 Operations

VPS Deployment & Monitoring

Why Go 24/7

Running OpenClaw on your laptop works until you close the lid. For a business operation — automated trading monitors, client reports, email sequences, market analysis — you need 24/7 uptime.

A VPS (Virtual Private Server) costs $5-20/month and runs your AI system around the clock.

Choosing a VPS Provider

Provider	Starting Price	Best For
DigitalOcean	$6/month	Simplicity, great docs
Hetzner	€4.50/month	Best value in Europe
Vultr	$6/month	Global locations
Linode	$5/month	Reliable, developer-friendly
AWS Lightsail	$5/month	AWS ecosystem

Recommended specs for OpenClaw:

- 2 GB RAM (minimum)
- 1 vCPU
- 50 GB SSD
- Ubuntu 22.04 LTS

Server Setup

1. Create and connect to your VPS:

```
ssh root@your-server-ip
```

2. Create a non-root user:

```
adduser openclaw
usermod -aG sudo openclaw
su - openclaw
```

3. Install Node.js:

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs
```

4. Install OpenClaw:

```
npm install -g openclaw
```

5. Transfer your configuration: Copy your local `~/.openclaw/` directory to the server:

```
scp -r ~/.openclaw/ openclaw@your-server-ip:~/.openclaw/
```

6. Start as a system service:

Create `/etc/systemd/system/openclaw.service`:

```
[Unit]
Description=OpenClaw AI Gateway
After=network.target

[Service]
Type=simple
User=openclaw
WorkingDirectory=/home/openclaw
ExecStart=/usr/bin/openclaw gateway start
Restart=always
RestartSec=10
Environment=NODE_ENV=production
```

```
[Install]
WantedBy=multi-user.target
```

Enable and start:

```
sudo systemctl enable openclaw
sudo systemctl start openclaw
sudo systemctl status openclaw
```

Monitoring

Check status remotely: Message your AI on Telegram — if it responds, it's running.

System health check cron: Set up a cron job that checks gateway health every 15 minutes and alerts you if it's down.

Log monitoring:

```
journalctl -u openclaw -f    # Live logs
journalctl -u openclaw --since "1 hour ago"    # Recent logs
```

Security Hardening

1. **SSH keys only** — Disable password authentication
2. **Firewall** — Allow only ports 22 (SSH) and 443 (HTTPS)
3. **Automatic updates** — Enable unattended-upgrades
4. **Fail2ban** — Block brute force attempts
5. **Regular backups** — Snapshot your VPS weekly

What You've Built

✅ VPS provisioned and configured ✅ OpenClaw running as a system service with auto-restart ✅ 24/7 uptime for all automations ✅ Remote monitoring via Telegram ✅ Security hardening applied ✅ A production-grade AI operations infrastructure

---

Congratulations. You've built a complete business automation system.

For the ultimate package — pre-built workspace, scraping engines, lead generation, email outreach, and 50+ cron templates ready to deploy — upgrade to Enterprise at erronatus.com.